

Virtualizzazione: VM e container

Isolamento delle risorse, densità, orchestrazione — linguaggio tipico delle prove PAT recenti.

Da ricordare

Container " " hypervisor multi-OS: il container non gestisce direttamente l'hardware e tipicamente non ospita SO guest diversi come una VM.

1. VM e hypervisor

L'hypervisor virtualizza l'hardware; ogni macchina virtuale esegue un sistema operativo guest completo.

La virtualizzazione server consente di eseguire più macchine virtuali (VM) su un unico host fisico. L'hypervisor (tipo 1 bare metal, direttamente sull'hardware, o tipo 2 hosted su un SO) astrae CPU, memoria, storage e rete, allocando risorse alle VM. Ogni VM ha un proprio SO guest, driver e applicazioni, con isolamento forte rispetto alle altre VM sullo stesso host.

Vantaggi: consolidamento hardware (meno server fisici), snapshot e backup a livello VM, clonazione rapida, live migration, laboratori e test isolati senza hardware dedicato. Costi: overhead di memoria e CPU rispetto al bare metal, licenze di hypervisor/guest, complessità di gestione e necessità di skill specifiche. È il fondamento di molti cloud IaaS.

In prova: definisci hypervisor e VM, cita isolamento e SO guest completo, confronta subito dopo con i container. Frase d'oro: VM = hardware virtuale + SO guest completo. Esempio PA: server di protocollo e di test sullo stesso blade, isolati come VM distinte.

Glossario

Hypervisor — Software che astrae l'hardware e ospita macchine virtuali.

SO guest — Sistema operativo eseguito all'interno di una macchina virtuale.

In prova: VM = hardware virtuale + SO guest completo.

Riferimenti: Programma tecnico virtualizzazione PAT

2. Container

Il container condivide il kernel dell'host e incapsula runtime e dipendenze dell'applicazione sopra il sistema operativo.

I container (ecosistema Docker e analoghi) isolano processi con namespace e cgroup del kernel Linux, impacchettando codice e dipendenze in immagini riproducibili. Non includono un kernel proprio: condividono quello dell'host. Partono in secondi, consumano meno risorse delle VM e favoriscono il principio «build once, run anywhere» (con le dovute configurazioni di ambiente).

Sono ideali per microservizi e pipeline CI/CD: lo stesso artefatto gira in test e produzione. Restano legati al kernel host: non si esegue tipicamente un Windows guest dentro un container Linux come si farebbe con una VM completa. L'isolamento è forte a livello processo, ma meno «assoluto» di una VM rispetto al kernel.

Frase d'esame: «container = isolamento applicativo su kernel condiviso; VM = SO guest completo su hypervisor». Insisti sul kernel condiviso: è il discriminante che la commissione cerca. Per l'assistente IT: saper leggere un Dockerfile e un docker-compose è valore operativo.

Glossario

Immagine container — Pacchetto immutabile con applicazione e dipendenze da cui si avviano i container.

Namespace / cgroup — Meccanismi del kernel Linux per isolamento di vista risorse e limiti di CPU/RAM/IO.

In prova: Insisti sul kernel condiviso: è il discriminante rispetto alle VM.

3. Vantaggi e rischi dei container

Leggerezza, portabilità e deploy rapido; attenzione alla sicurezza dell'host condiviso e della supply chain delle immagini.

Vantaggi: densità elevata (più servizi per host), avvio rapido, portabilità tra ambienti, scaling granulare dei servizi, allineamento DevOps e rollback veloci tramite versionamento delle immagini. Consentono di standardizzare il runtime e ridurre il «funziona solo sulla mia macchina».

Rischi: fuga dal container verso l'host se il kernel o il runtime sono vulnerabili; immagini non aggiornate o provenienti da registry non fidati; segreti (password, token) cablati nelle immagini; lateral movement tra container mal segmentati sulla stessa rete. Mitigazioni: scanning delle immagini (CVE), least privilege (non root), network policy, aggiornamento runtime, registry interni firmati, secret manager.

In sintetica bilancia 2 vantaggi e 2 rischi con una mitigazione ciascuno: dimostra consapevolezza operativa da PA, non entusiasmo tecnologico acritico. Non dire solo «i container sono meglio»: mostra rischi di kernel condiviso e supply chain.

Glossario

Supply chain — Catena di fornitura del software: base image, dipendenze, registry, build pipeline.

Image scanning — Analisi automatica delle immagini alla ricerca di vulnerabilità note.

In prova: Non solo «i container sono meglio»: mostra rischi di kernel condiviso e supply chain.

4. Orchestrazione

Kubernetes e analoghi schedulano, scalano e ripristinano container su un cluster.

Quando i container crescono di numero serve un orchestratore: schedula i workload sui nodi del cluster, gestisce service discovery, bilanciamento, rolling update, autoscale e self-healing (riavvio dei pod falliti secondo lo stato desiderato). Kubernetes è lo standard de facto; esistono anche orchestratori gestiti in cloud (servizi PaaS/CaaS) che riducono l'onere del control plane.

Per la PA l'orchestrazione porta vincoli di competenza, osservabilità (metriche, log, tracing) e sicurezza (RBAC, network policy, secret management, admission control). Non è obbligatoria per ogni applicativo: un piccolo servizio può vivere su VM o su PaaS senza cluster dedicato; forzare Kubernetes ovunque aumenta costi senza benefici.

Risposta d'esame: orchestrazione = scheduling + scaling + self-healing; esempio Kubernetes; citare che aumenta complessità gestionale. Tre parole bastano se spiegate bene. Collega a scale-out: l'orchestratore è lo strumento naturale per aggiungere/rimuovere repliche.

Glossario

Self-healing — Capacità del cluster di ripristinare workload non sani secondo lo stato desiderato.

Kubernetes — Orchestratore open source de facto per container su cluster di nodi.

In prova: Associa Kubernetes a scheduling, scaling e self-healing — tre parole bastano.

5. Scale-out e scale-up

Scalabilità verticale = più risorse a un nodo; orizzontale = più nodi/istanze.

Scale-up (verticale): si aumentano CPU/RAM/disco del server o della VM. È concettualmente semplice ma ha un tetto hardware, può richiedere downtime per il resize e concentra il rischio su un unico nodo. Scale-out (orizzontale): si aggiungono istanze dietro un load balancer; tipico dei servizi stateless e dei container orchestrati, consente di crescere in modo più elastico.

Le applicazioni stateful (database monolitici tradizionali) scalano orizzontalmente con più difficoltà e richiedono sharding, replica o cluster specifici. In cloud il scale-out si combina con autoscaling basato su metriche (CPU, code, latenza). Per la PA: progettare servizi stateless dove possibile facilita continuità e migrazione cloud.

Quiz: «Aggiungo un altro server web dietro al bilanciatore» !' scale-out. «Passo da 8 a 32 GB di RAM sulla stessa VM» !' scale-up. Schema: verticale = più grosso; orizzontale = più numerosi. Collega a BC/DR: più nodi possono aumentare resilienza se distribuiti correttamente.

Glossario

Scale-up — Scalabilità verticale: più risorse sullo stesso nodo.

Scale-out — Scalabilità orizzontale: più nodi/istanze dietro bilanciatore.

In prova: Verticale = più grosso; orizzontale = più numerosi.

6. Business continuity, DR, RPO e RTO

BC = continuità operativa; DR = ripristino dopo disastro; RPO e RTO quantificano perdita dati accettabile e tempo di ripristino.

Business continuity (BC) comprende processi organizzativi e tecnologie per mantenere i servizi essenziali anche in caso di incidenti (piano, ruoli, comunicazioni, modalità degradate). Disaster recovery (DR) è il sottoinsieme orientato al ripristino dopo eventi gravi (sito down, ransomware esteso, disastro fisico), spesso su sito secondario, cloud o backup immutabili.

RPO (Recovery Point Objective): massima perdita di dati tollerabile in termini di tempo (es. «al più gli ultimi 15 minuti» !' backup/replica almeno ogni 15 minuti). RTO (Recovery Time Objective): tempo massimo accettabile per ripristinare il servizio dopo l'evento. Backup, replica sincrona/asincrona e esercitazioni di restore devono essere allineati a RPO/RTO definiti dall'ente, non scelti «a sentimento».

In prova PAT: definisci BC vs DR, poi RPO vs RTO con un esempio numerico semplice. Evita di usarli come sinonimi. Schema mnemonico: RPO = «punto nel passato» (quanto perdo); RTO = «quanto ci metto a ripartire». Un backup non testato non garantisce nessun RTO reale.

Glossario

RPO — Recovery Point Objective: quanti dati (in tempo) posso perdere al massimo.

RTO — Recovery Time Objective: quanto tempo posso impiegare per tornare operativi.

In prova: RPO = «punto nel passato»; RTO = «quanto ci metto a ripartire».

Riferimenti: Piani BC/DR ente · Buone pratiche backup